

Exam

Algorithmic Thinking
ALD (MTD.ba VZ)
SS 2026

Surname(s):

Given name(s):

Student ID / Mat.-Nr.:

	maximum	achieved
Question 1	20	
Question 2	20	
Question 3	18	
Question 4	12	
Question 5	18	
Question 6	12	
Total:	100	

Time allowed: 60 minutes

DO NOT TURN OVER UNTIL INSTRUCTED!

Question 1 (20 points) You want shapes in your drawing app to be drawable and savable, and you are deciding between an abstract class and interfaces.

Explain:

- The difference between an interface and an abstract class (state, constructors, multiple implementation)
- Why a class can implement `Drawable`, `Saveable` but cannot extend two classes
- What “programming to an interface” means and why `Drawable[] items` is more flexible than `Circle[] items`
- What a default method in an interface is, and what the “diamond problem” is when two interfaces define the same default method

Question 2 (20 points) A small game stores players and scores in a database. Explain how a 3-layer architecture (Presentation, Business Logic, Data Access) organises this code.

- What is the responsibility of each layer, with a concrete example for each?
- State the “golden rule” about which layer may call which
- Why should there be *no* SQL in the model classes and *no* JDBC outside the data-access layer?
- Where does a business rule like “a negative score is clamped to 0” belong, and why is that *not* the DAO’s job?

Question 3 (18 points) Tree traversals produce different orderings, and one of them is special for binary search trees (BSTs).

- Describe inorder, preorder, and postorder traversal
- Which traversal of a BST yields the values in *sorted* order, and why?
- Give a typical use for preorder (copy/serialise) and for postorder (deletion / expression evaluation)
- Sketch the recursive pattern shared by all three (base case + recurse left/right + visit)

Question 4 (12 points) Your client uses Java's `HttpClient` and Jackson to talk to the leaderboard.

- What roles do `HttpClient`, `HttpRequest`, and `HttpResponse` play?
- How does `ObjectMapper` convert between JSON text and Java objects (serialise vs deserialise)?
- Why is it useful to configure the mapper to ignore unknown JSON properties when consuming a third-party API?

Question 5 (18 points) Explain the difference between composition (“has-a”) and inheritance (“is-a”), and why “favour composition over inheritance” is common advice.

- Give a clear “has-a” example (e.g. a Player *has a* Weapon)
- Give a clear “is-a” example (e.g. a Circle *is a* Shape)
- Explain how you decide which relationship to use when modelling a problem
- Give an example where someone uses inheritance but composition would have been the better choice, and explain why

Question 6 (12 points) You implement an inventory as a `List<Item>` (each `Item` has a name, weight, and value) and need to: remove all items over a weight limit, then list the remaining items sorted by value.

Explain how these concepts work together:

- **Collections:** why you use `removeIf` instead of removing inside a for-each loop
- **Generics:** what `List<Item>` gives you over a raw `List`
- **Ordering:** how a `Comparator` sorts by value, and why `equals` on `Item` matters for removal by object

